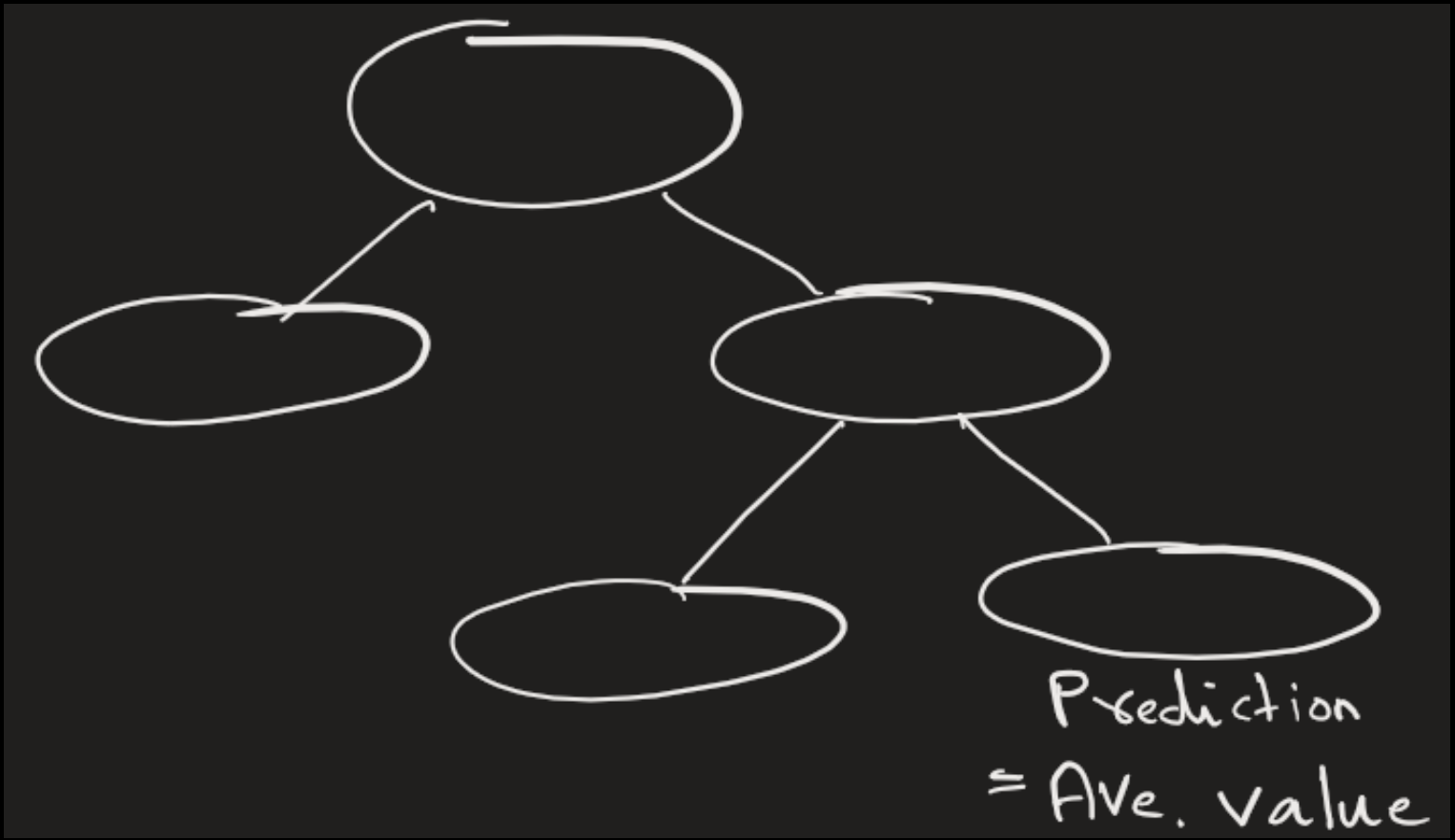
Decision Tree Regressor

A **Decision Tree** is a supervised machine learning algorithm used for **classification and regression**.

Decision Tree Regression

Idea:
Splits data into regions and predicts average value per region.



Problem Statement

We want to **predict house price** based on:

- Size (sqft)
- Number of rooms
- Age of house
- Distance from city center

We want output: **House Price (continuous value)**

Therefore, this is **Regression Problem**

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

Here it should be noted that

- Larger size → higher price
- Less distance → higher price
- Newer houses → costly

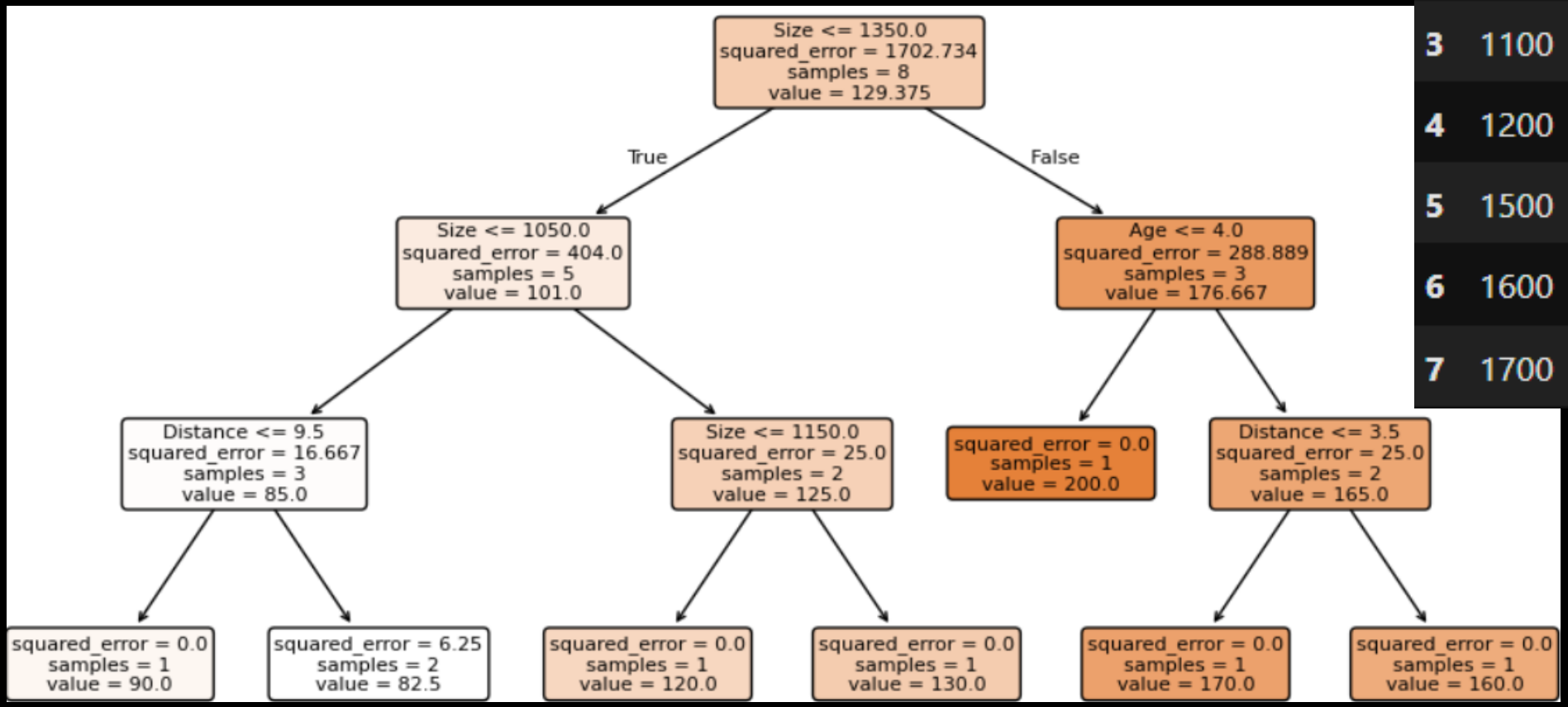
	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

Here you make series of decision splits based on features and their values that gives you **maximum information gain.**

<u>This is discussed in detail in DT classifier.</u>

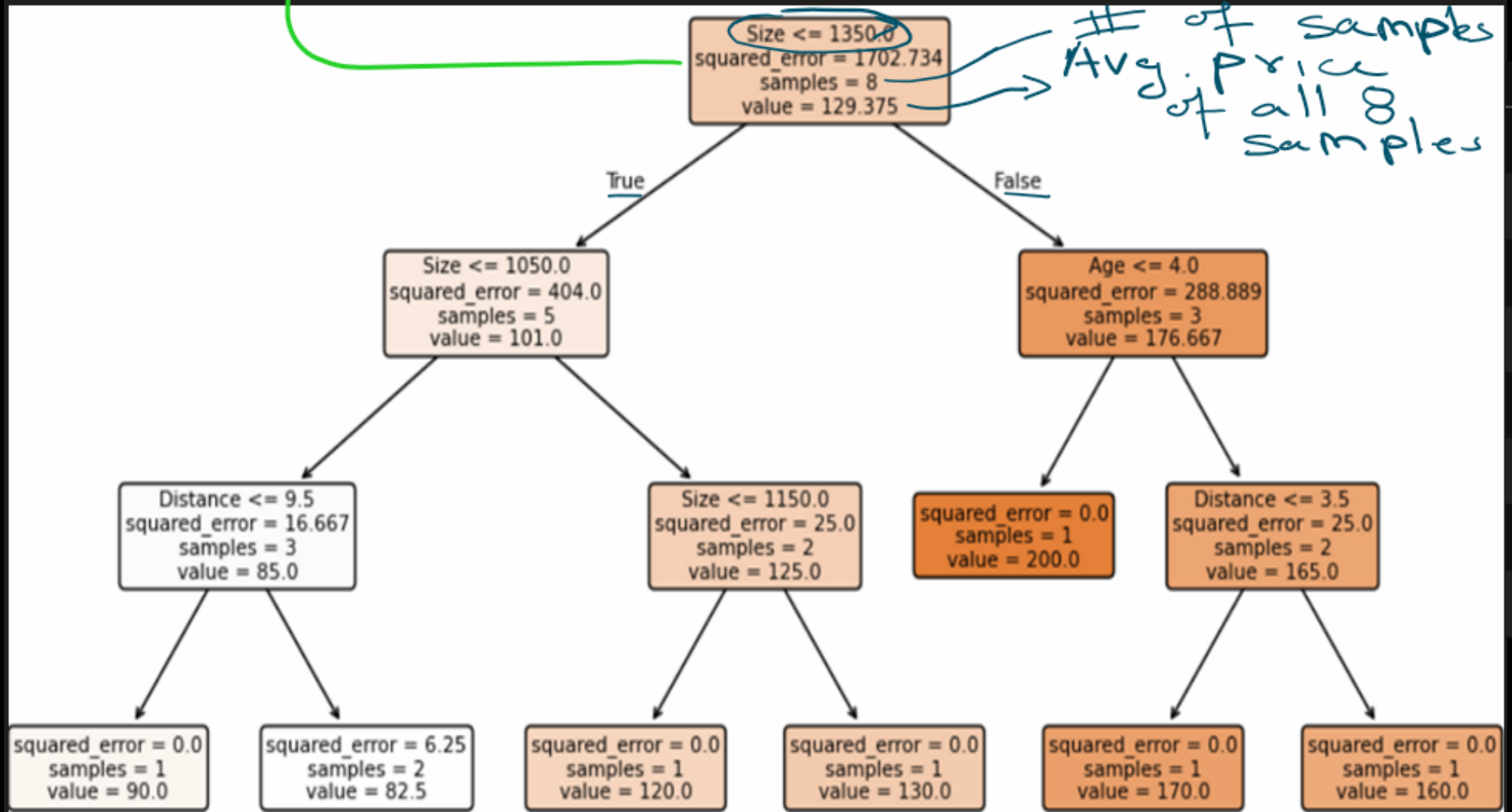
At the end you get final DT shown below.

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200



What does each value mean ?
Let's look at root node

$$S.E = \frac{(80-129.3)^2 + (85-129.3)^2 + \cdots + (200-129.3)^2}{8}$$



# of samples

Avg. price of all 8 samples

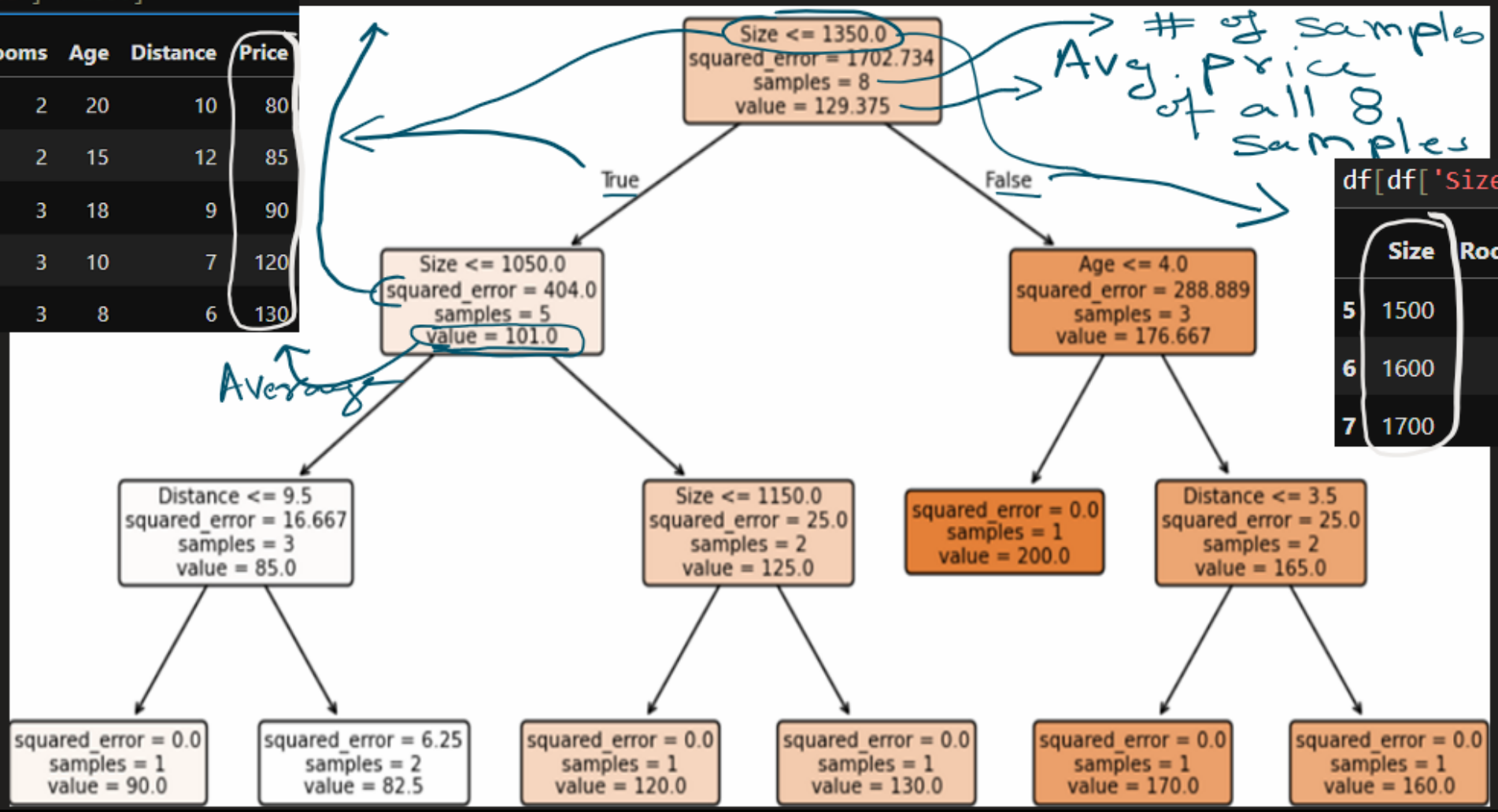
	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

Data split at root node Size <= 1350

$$SE = \frac{(80-101)^2 + (85-101)^2 + \cdots + (130-101)^2}{5}$$

```
df[df['Size'] <=1350]
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130




```
df[df['Size'] > 1350]
```

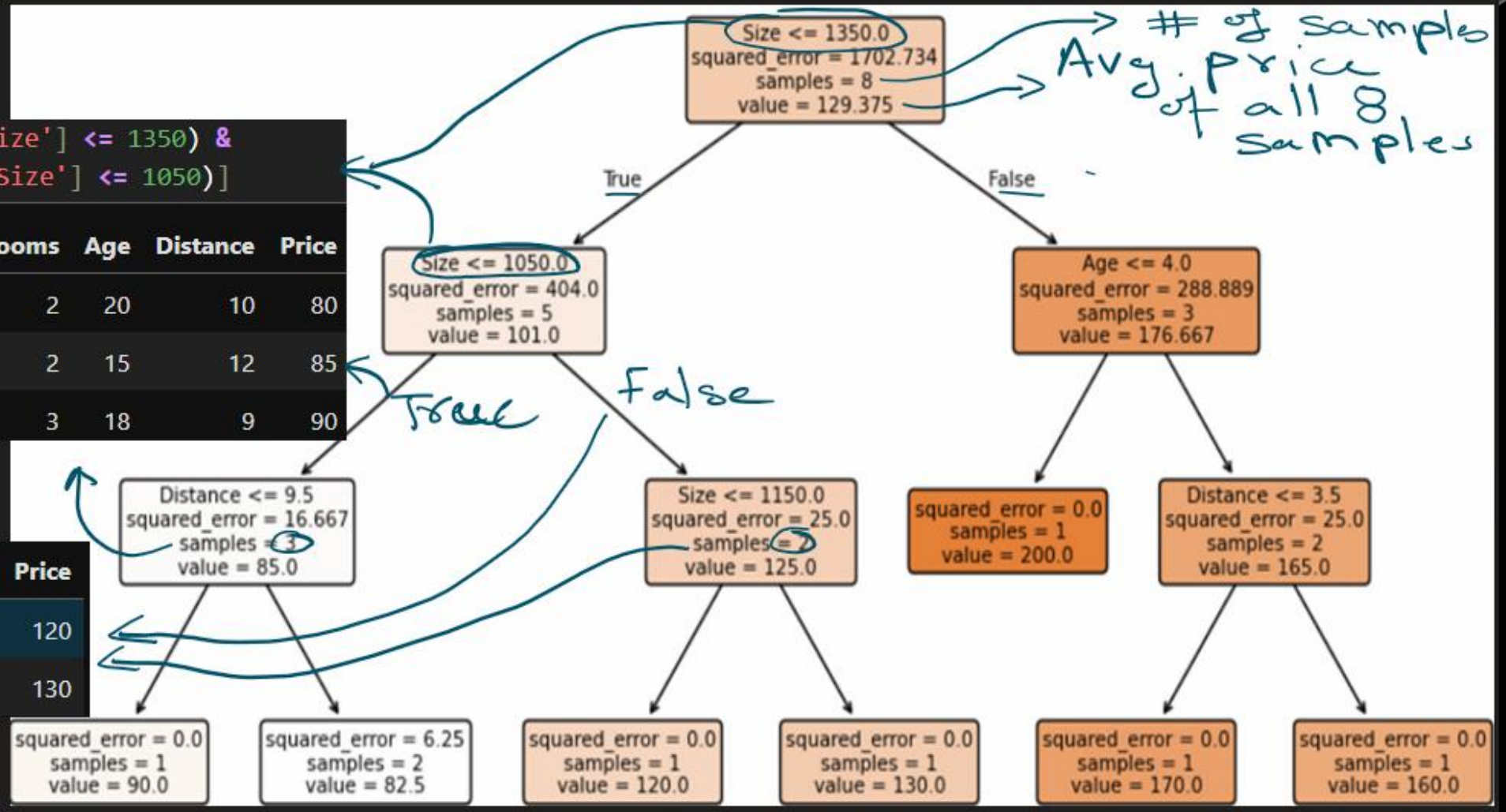
	Size	Rooms	Age	Distance	Price
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

Data split at node Size<=1050

```
df[(df['Size'] <= 1350) &
    (df['Size'] <= 1050)]
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90

	Size	Rooms	Age	Distance	Price
3	1100	3	10	7	120
4	1200	3	8	6	130



Data split at node Distance <= 9.5

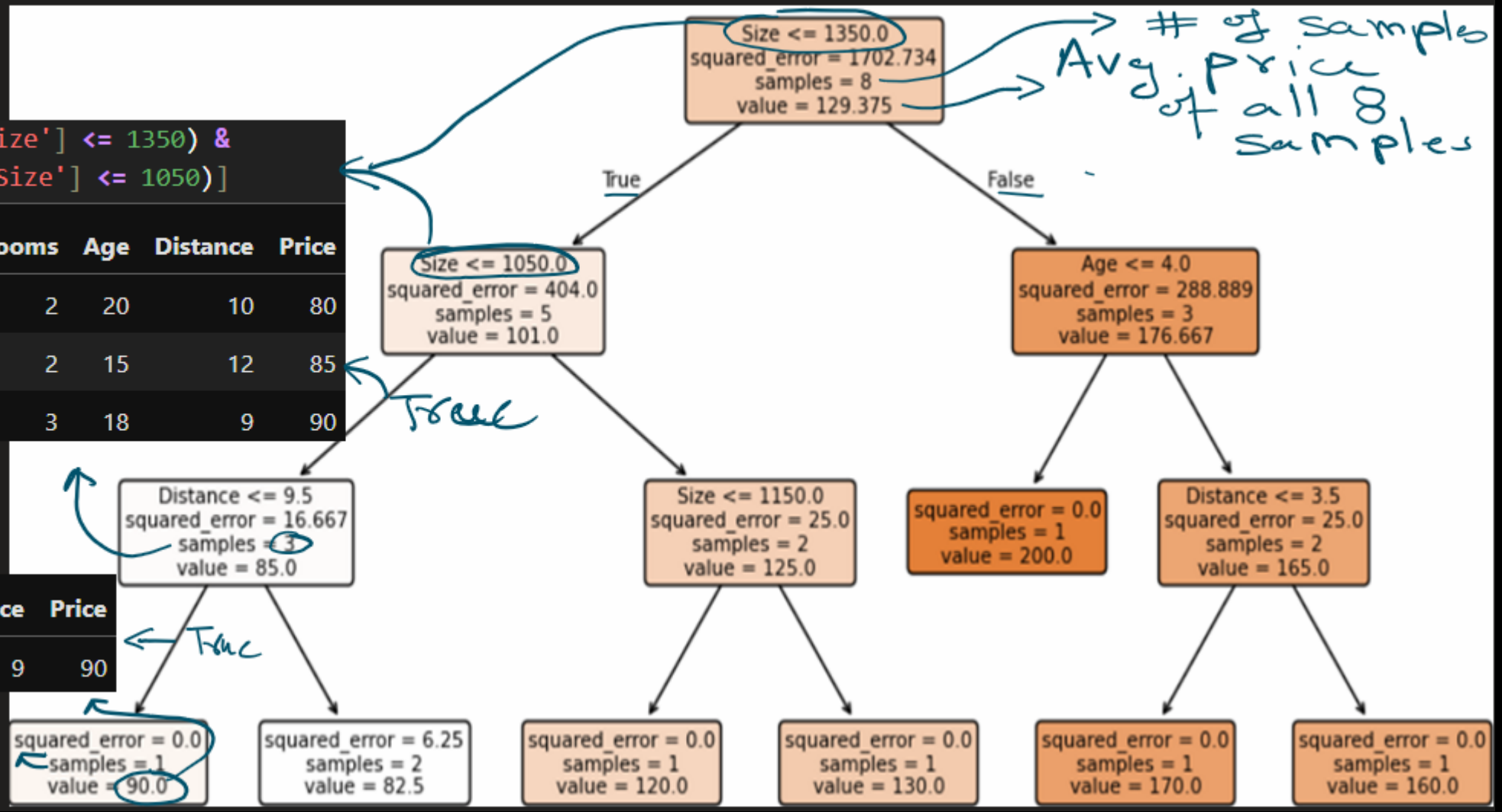
```
df[(df['Size'] <= 1350) &
   (df['Size'] <= 1050)]
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90

True

	Size	Rooms	Age	Distance	Price
2	1000	3	18	9	90

True



# of samples

Avg. price of all 8 samples

Predict house price (1400, 3, 6, 5):

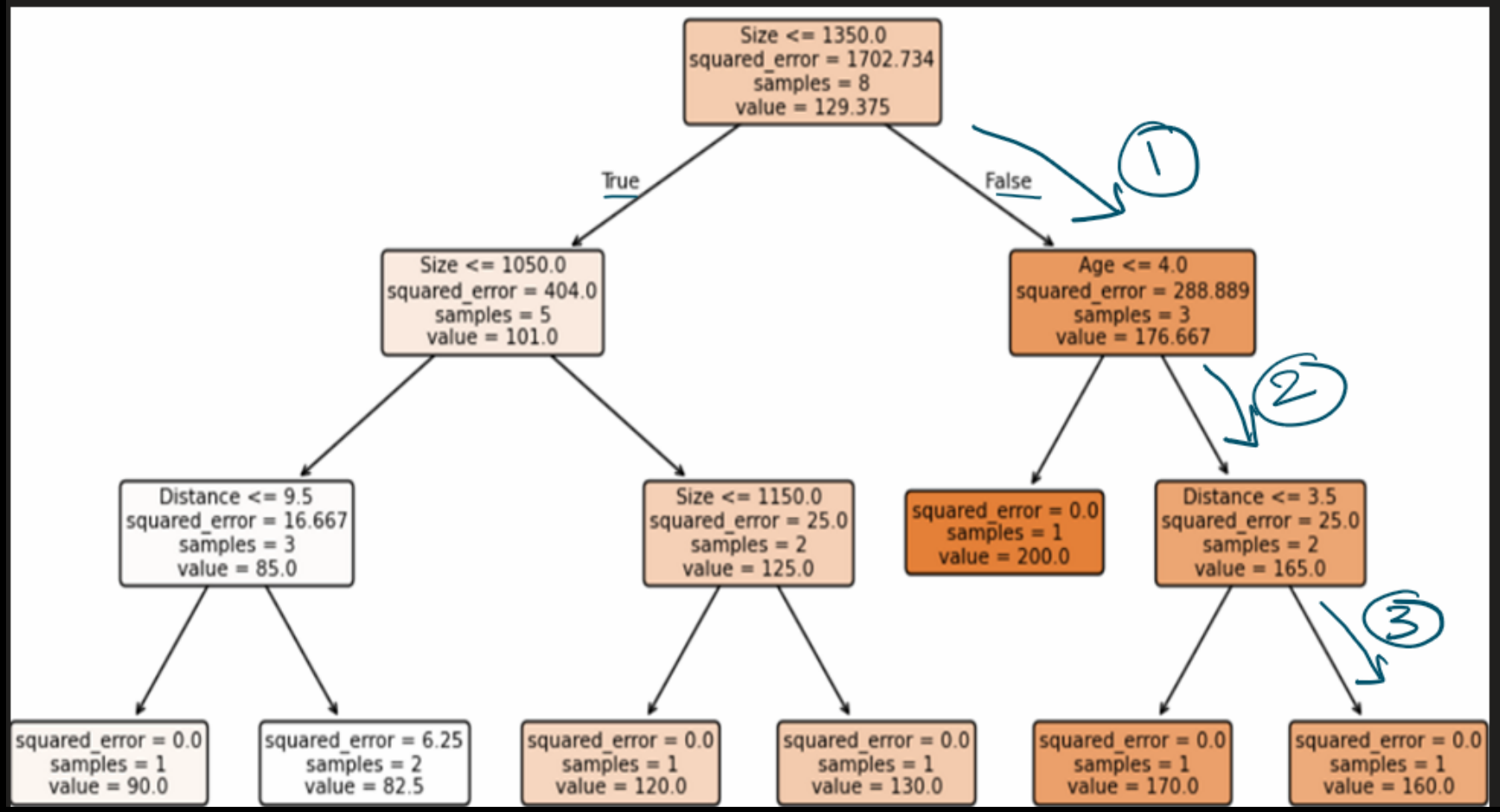
1) Is size <= 1350 -> No
2) Is Age <=4 -> No
3) Is Distance <=3.5 -> No

Now you have reached leaf node

Calculate the average of samples in leaf node: 160

Predict price (1400, 3, 6, 5)

size, room, Age, Distance



The python code:

```python
# Import required libraries
import pandas as pd
import numpy as np
from sklearn.tree import DecisionTreeRegressor
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

```python
# Create a simple dataset

data = {
    "Size": [800, 900, 1000, 1100, 1200, 1500, 1600, 1700],
    "Rooms": [2, 2, 3, 3, 3, 4, 4, 5],
    "Age": [20, 15, 18, 10, 8, 5, 7, 3],
    "Distance": [10, 12, 9, 7, 6, 4, 3, 2],
    "Price": [80, 85, 90, 120, 130, 160, 170, 200]
}

df = pd.DataFrame(data)
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

The python code:

```python
# Features (X) and Target (y)

X = df[["Size", "Rooms", "Age", "Distance"]]
y = df["Price"]
```

```python
# Create model
model = DecisionTreeRegressor(max_depth=3, random_state=42)

# Train model
model.fit(X, y)

# Explain:
# max_depth=3 → keeps tree small for visualization
# Tree learns rules, not equations.
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

The python code:

```python
# Predict price of a new house
new_house = [[1400, 3, 6, 5]]
# convert it to DF
new_house_df = pd.DataFrame(new_house, columns=X.columns)

predicted_price = model.predict(new_house_df)

print("Predicted Price:", predicted_price[0])
```

```
Predicted Price: 160.0
```

	Size	Rooms	Age	Distance	Price
0	800	2	20	10	80
1	900	2	15	12	85
2	1000	3	18	9	90
3	1100	3	10	7	120
4	1200	3	8	6	130
5	1500	4	5	4	160
6	1600	4	7	3	170
7	1700	5	3	2	200

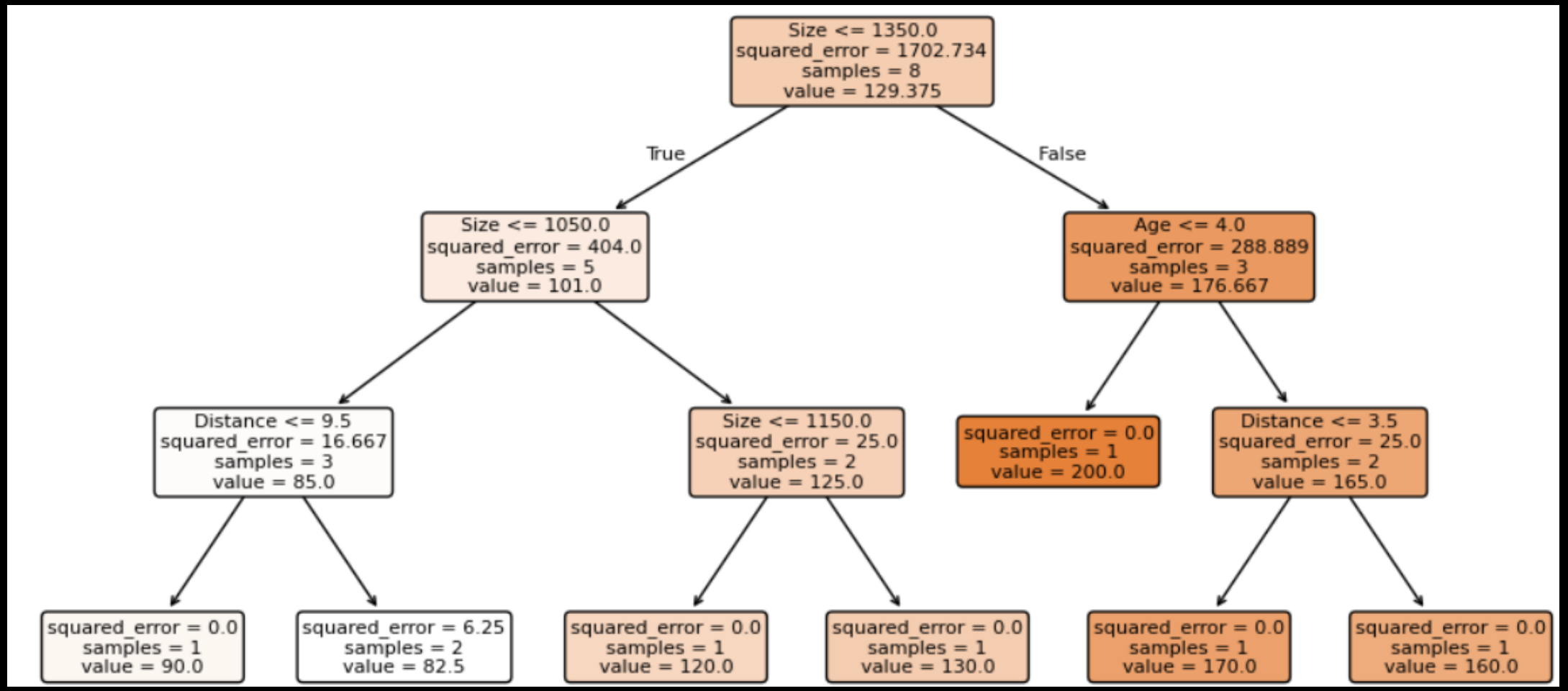
The python code:

```
plt.figure(figsize=(12,6), )

plot_tree(
    model,
    feature_names=X.columns,
    filled=True,
    rounded=True
)

plt.show()
```

The python code:



Advantage	Explanation
No scaling needed	Works on raw data. No need to scale
Nonlinear learning	Captures complex patterns
Easy interpretation	Rule-based are easy to understand
Handles mixed features	Numeric + categorical

Use Cases

- Real estate pricing
- Risk modeling

EXTRA

See jupyter notebook for code